L-Expr: ANTLR-Visitoren und Rückgabewerte

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Erinnerung: ANTLR-Toolchain: Von String zum Parse-Tree

```
var source = "(+ 1 2)";

var input = CharStreams.fromString(source);
var lexer = new LispyLexer(input);
var tokens = new CommonTokenStream(lexer);

var parser = new LispyParser(tokens);
var tree = parser.lexpr();
```

Ergebnis: ein **Parse-Tree**, noch kein AST

ANTLR: Generierte Klassen

- Grammatik: `Lispy.g4`
  - mit Regel `eform : '(' '+' eform eform+ ')'   #addExpr ;`

- ANTLR generiert:
  - `LispyLexer`, `LispyParser`
  - `LispyBaseVisitor<T>` mit

```
T visitAddExpr(LispyParser.AddExprContext ctx) {
    return visitChildren(ctx);
}
```

AST-Builder: Beispiel Int-Literal und Addition

```java
public class LispyAstBuilderLexpr extends LispyBaseVisitor<Object> {

    @Override
    public Expr visitLexpr(LexprContext ctx) { return asExpr(visit(ctx.eform())); }

    @Override
    public Expr visitIntLit(IntLitContext ctx) {
        int value = Integer.parseInt(ctx.INT().getText());
        return new IntLiteral(value);
    }

    @Override
    public Expr visitAddExpr(AddExprContext ctx) {
        List<Expr> ops = ctx.eform().stream()
                            .map(this::visit)
                            .map(this::asExpr).toList();
        return ops.stream().reduce(AddExpr::new).orElseThrow();
    }
}
```

ANTLR-Pipeline komplett

```
var source = "(+ 1 2)";

var input = CharStreams.fromString(source);
var lexer = new LispyLexer(input);
var tokens = new CommonTokenStream(lexer);

var parser = new LispyParser(tokens);
var tree = parser.lexpr();

var builder = new LispyAstBuilderLexpr();
var ast = builder.visitLexpr(tree);
```

Default-Implementierung: Wann reichen die Default-`visitXYZ()`-Methoden?

Default in `LispyBaseVisitor<T>`:

```
T visitAddExpr(AddExprContext ctx) { return visitChildren(ctx); }
```

```
public T visitChildren(RuleNode node) {
    return node.childs().stream()
            .map(c -> c.accept(this))
            .reduce((left, right) -> right).orElse(null);
}
```

- **Überschreiben**, wenn:
  - Semantik / Auswertung benötigt: Beispiel `visitAddExpr()`
  - Standardlogik falsch: `parens : '(' expr ')' ;`
- **Defaultverhalten** reicht, wenn
  - die Regel nur "durchgereicht" werden soll

```
expr    : addExpr ;
addExpr : multExpr (('+' | '-') multExpr)* ;
```

Wrap-up

- ANTLR-Visitoren sind generisch: `<T>` → Rückgabetyp aller `visitXYZ()`
- `BaseVisitor` implementiert `visitChildren()`
- Kontextobjekte (`...Context`) kapseln Bestandteile einer Grammatikregel
- Benannte Alternativen (`#label`) erzeugen eigene Kontextklassen bzw. Zugriffsmethoden

- Faustregel:
  - Für **semantisch relevante** Regeln `visitXYZ()` überschreiben
  - Für reine **Durchleitungs-/Klammer-Regeln** kann der Default genügen



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: *de3ef50 2026-11-14 lexpr: improve wording on visitChildren()*